Parallel & Distributed Computing

INTRODUCTION TO PARALLEL AND DISTRIBUTED SYSTEMS

"WHY USED & WHY NOT USED"

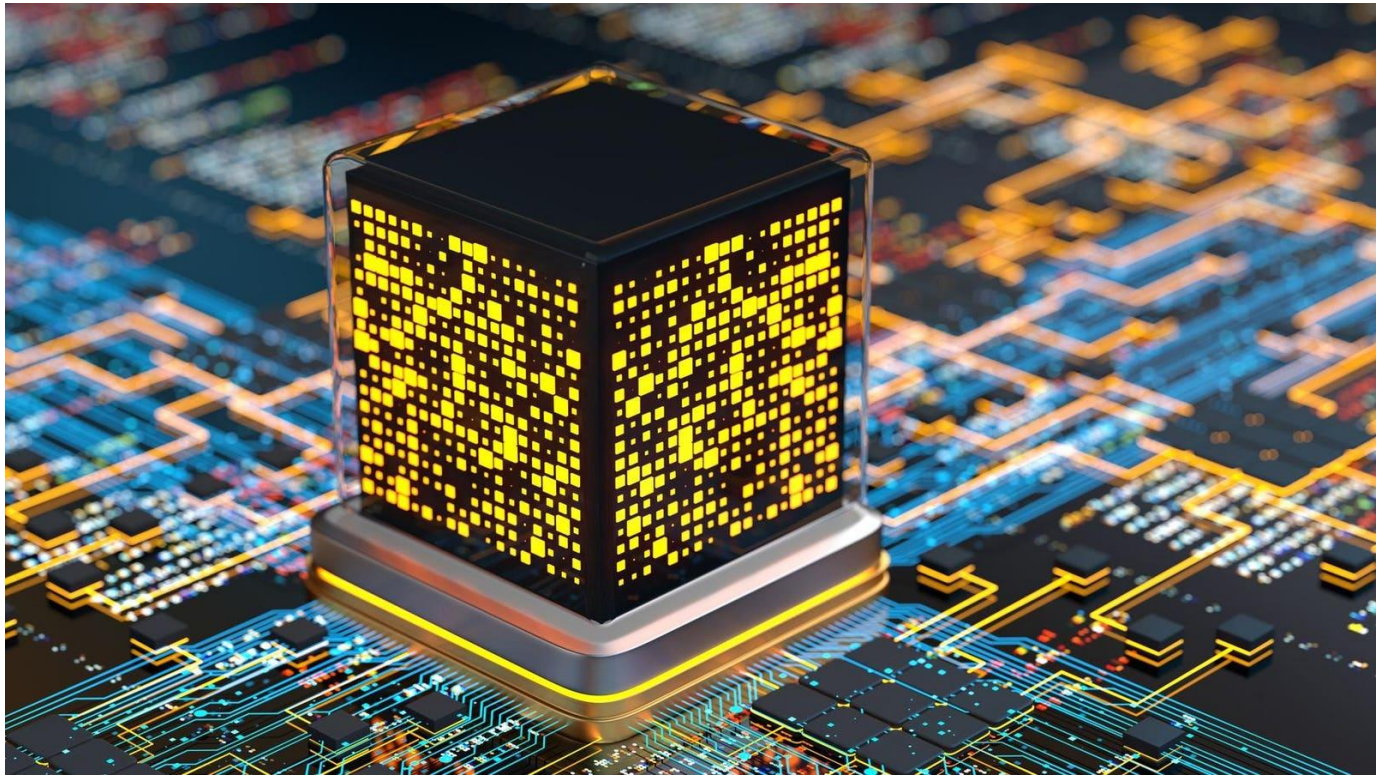
Dr. Muzamil Ahmed (Assistant Professor)
Namal University Mianwali

Lesson from Quran

Lecture Outline

- Computing
- What is Parallel Computing?
- Multi-core vs Multi-processor Systems
- What is Distributed Computing?
- Parallel vs Distributed Computing
- Why Use Parallel & Distributed Computing?
- Why Not Used – Challenges
- Distributed System Trade-offs
- Summary

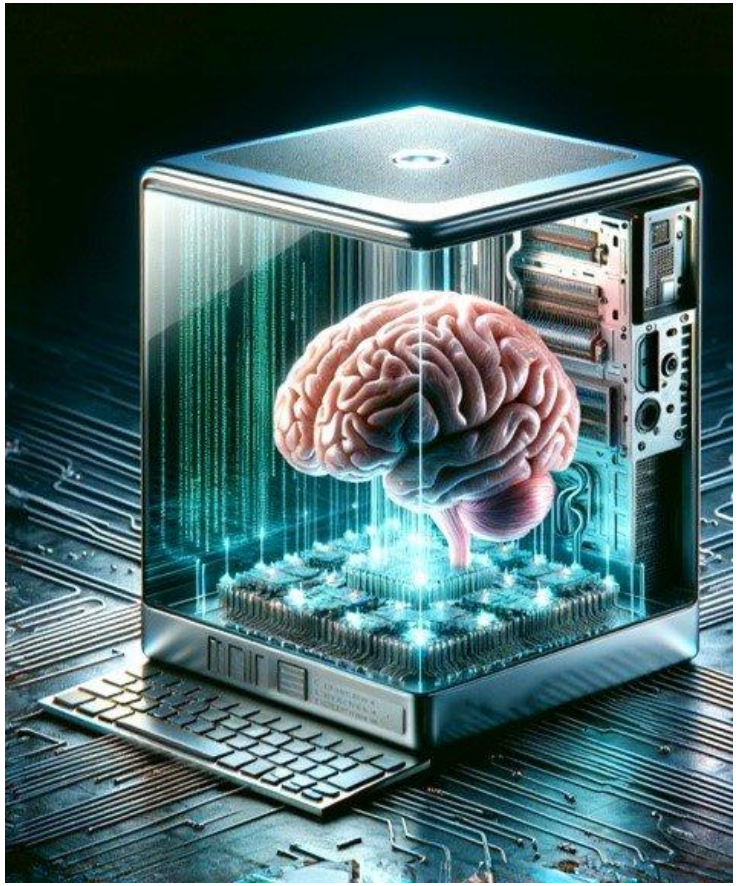
Parallelism is not just faster computing, it’s smarter computing



How many cores does your laptop/phone have? Do you know if they are being fully utilized?

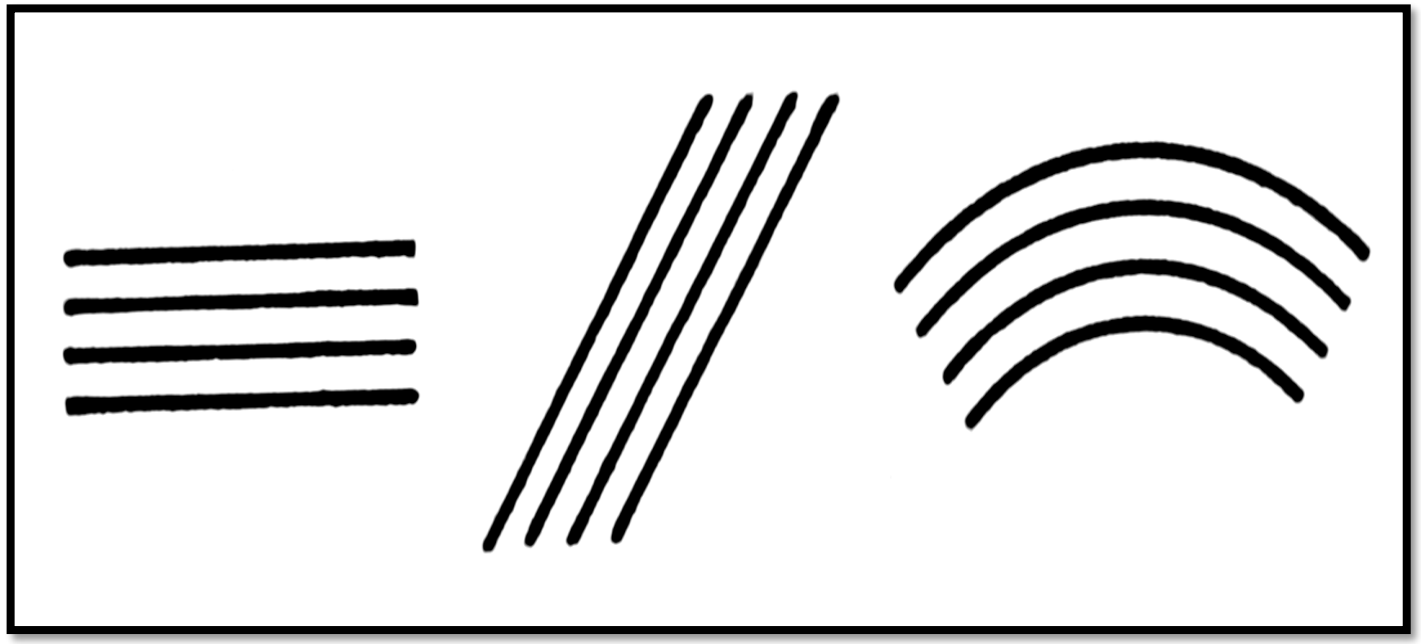
Computing

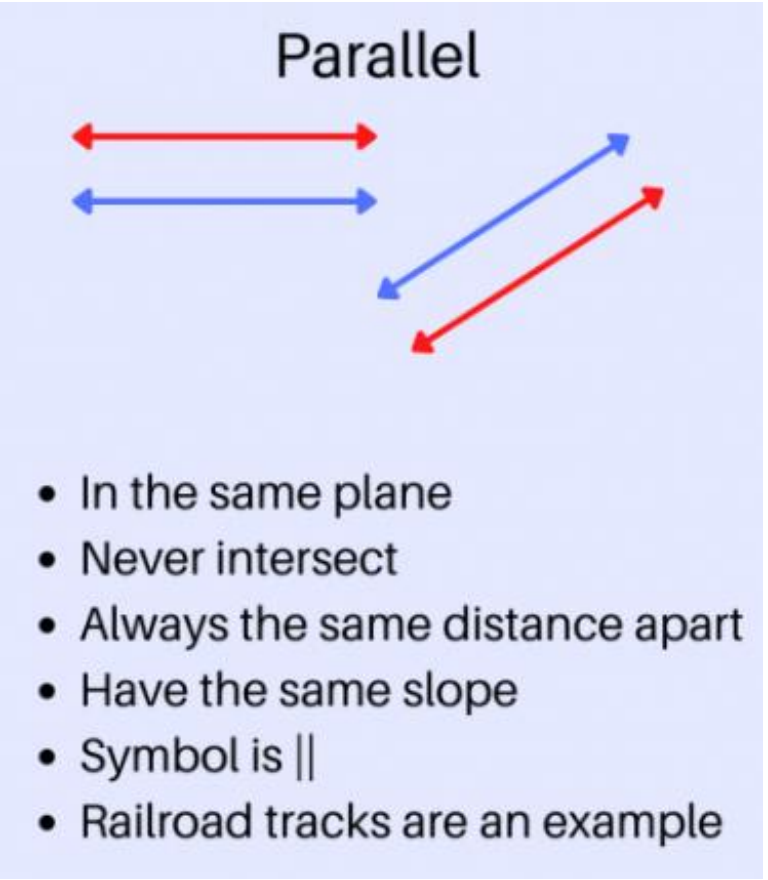
- Computing is the process to complete a given goal-oriented task by using computer technology.
- Computing may include the design and development of software and hardware systems for a broad range of purposes, often consist of structuring, processing and managing any kind of information.



Parallel

- Parallel (in mathematics) means two lines that never intersect — think of an equal sign.

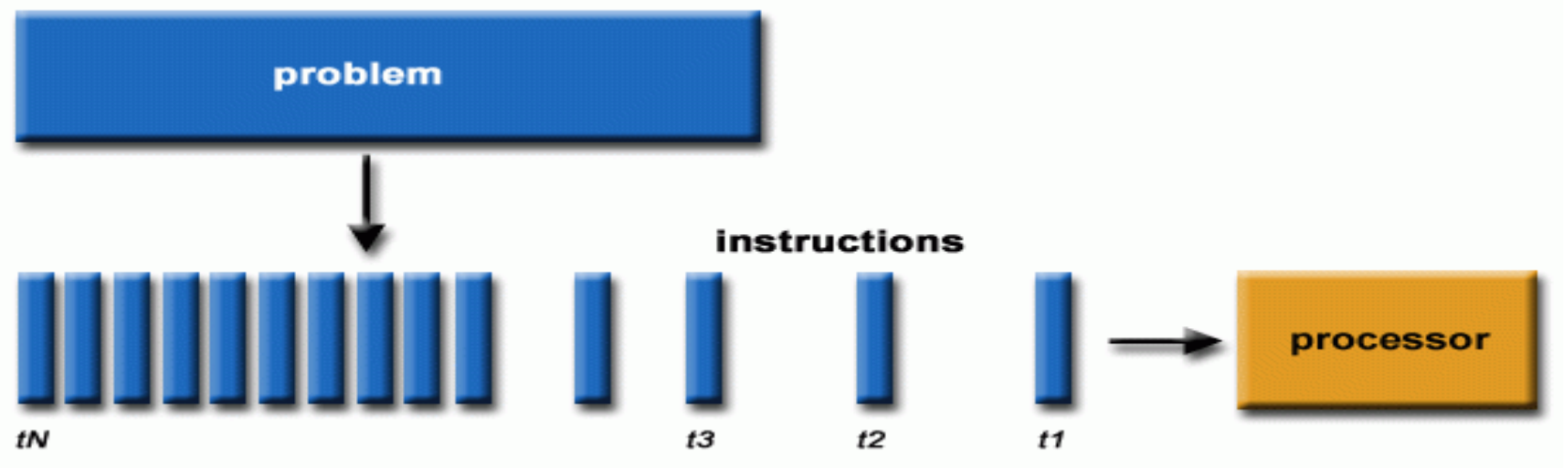


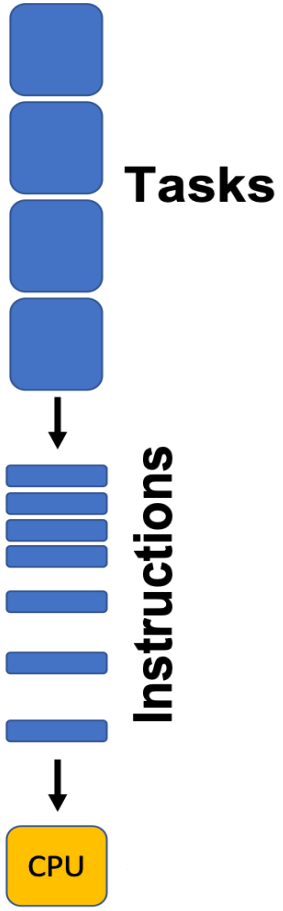


What is Parallel Computing?

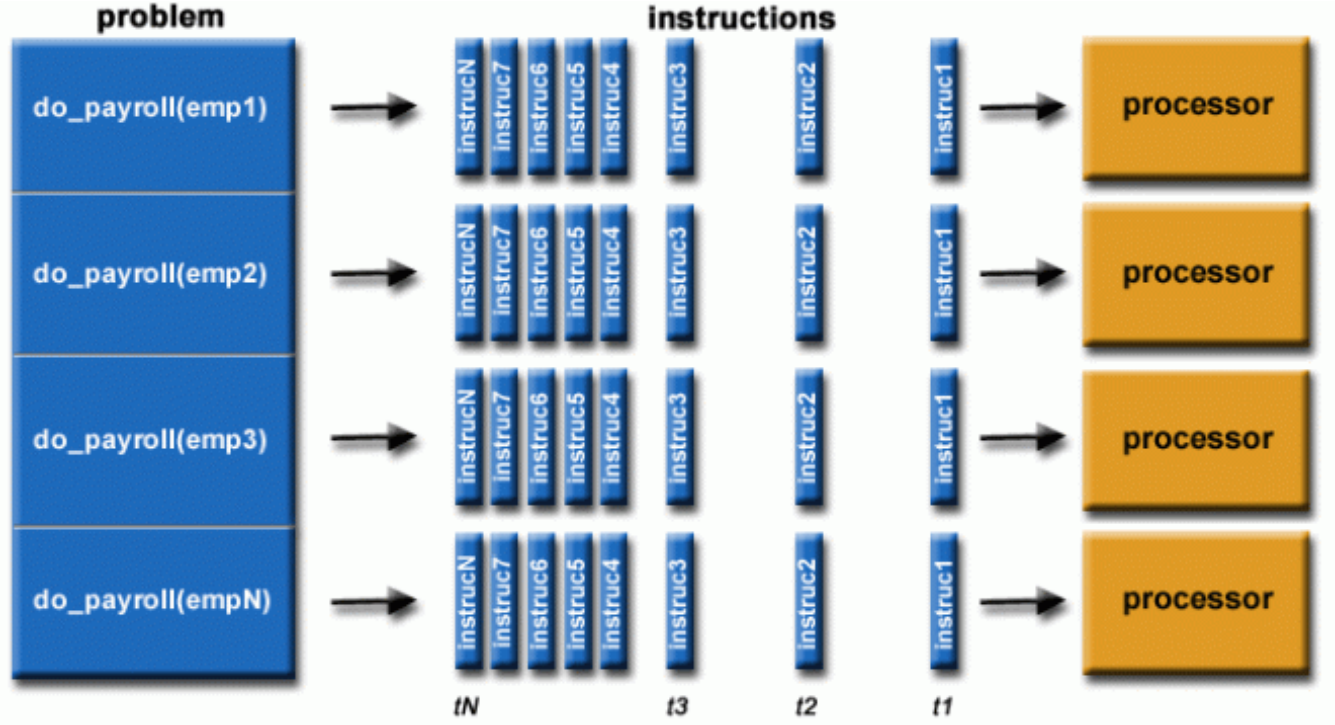
- Multiple computations executed simultaneously on shared memory or a single machine.
- Divide a large problem into smaller sub-tasks.
- Each task computed independently or in synchronization.
- Modern Examples:
  - Training neural networks on multiple GPUs.
  - Rendering graphics in AAA games.
  - Scientific simulations like climate modeling.
- Can all computations be split evenly?
- How do you manage dependencies between tasks?

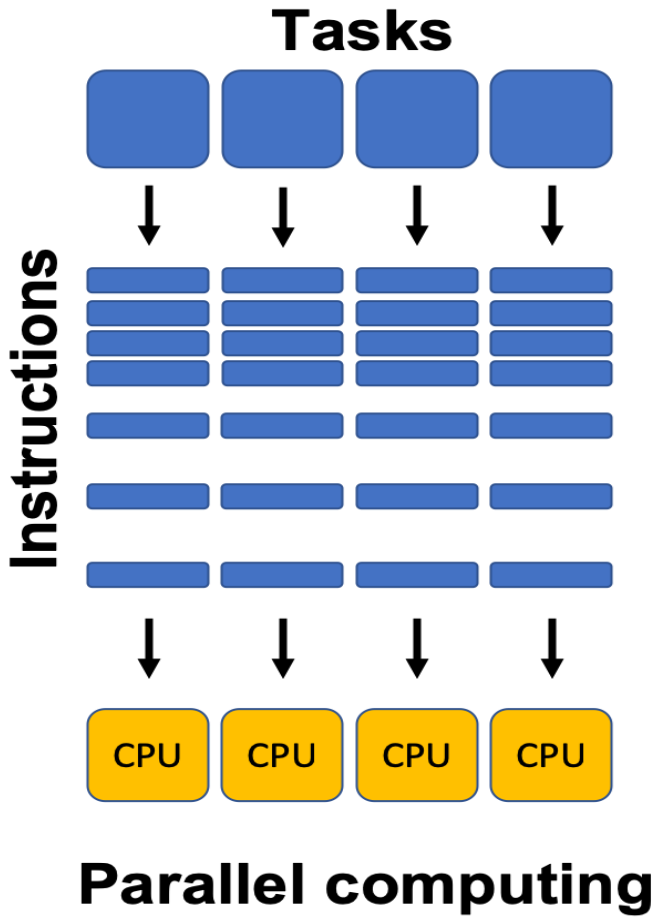
What is Parallel Computing?





What is Parallel Computing?

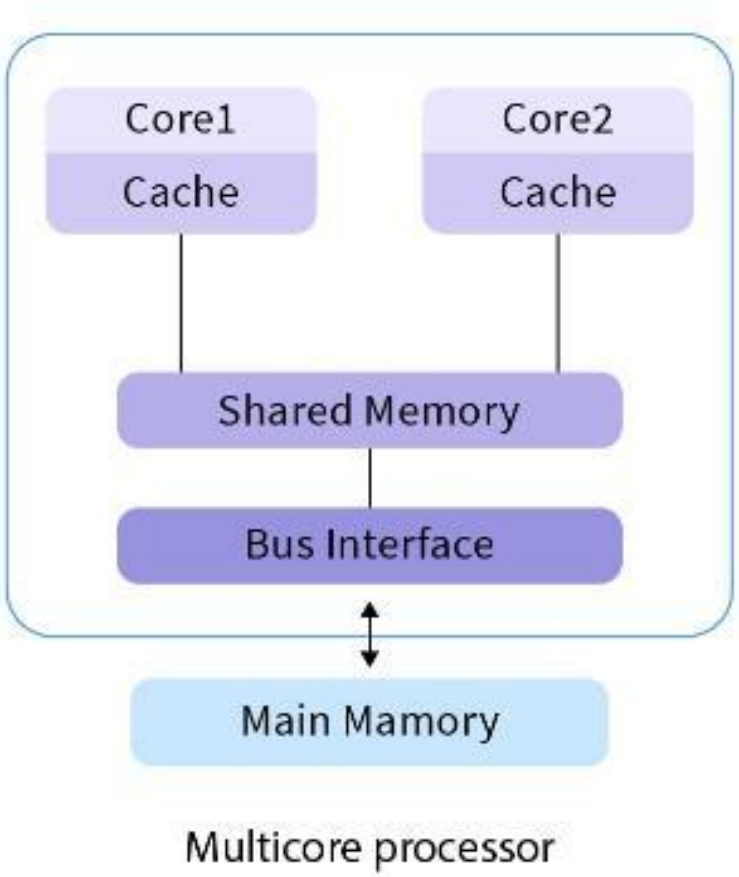




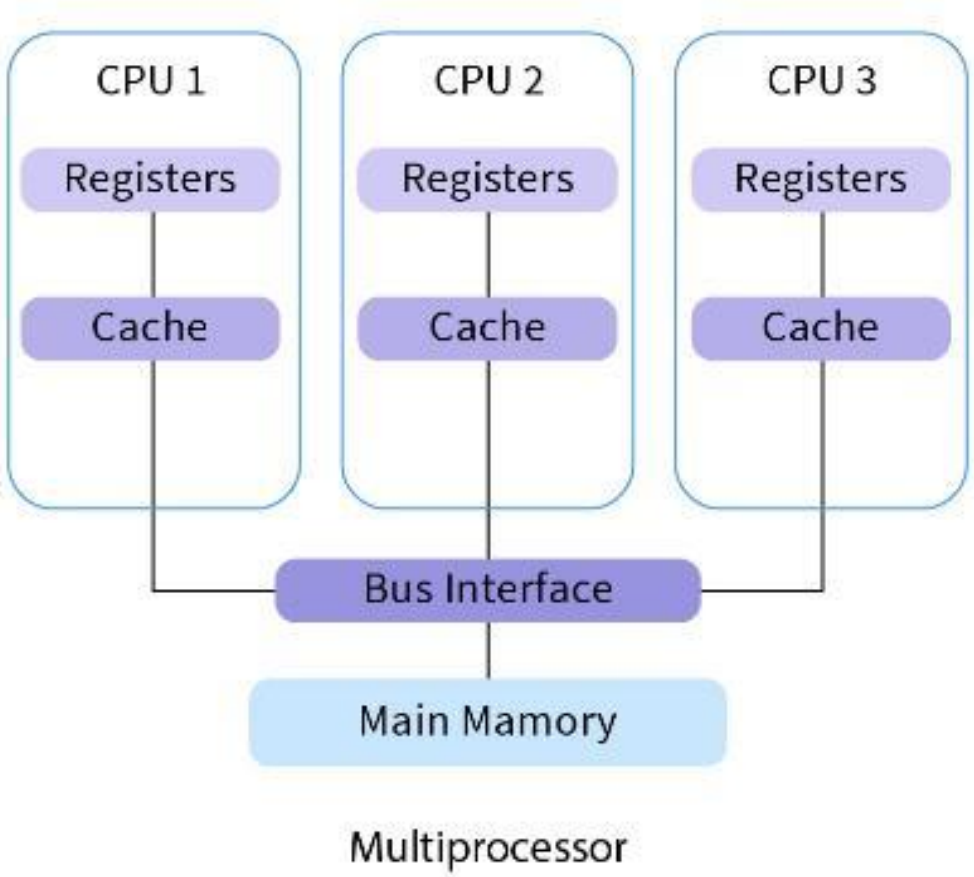
Multi-core vs Multi-processor Systems

- **Multi-core Processor:**
  - A **single CPU chip** with multiple processing cores.
  - Cores share memory & cache (sometimes).
  - Example: Intel i7 with 8 cores.
  - **Use Case:** Laptops, smartphones, desktops → efficient multitasking.
- **Multi-processor System:**
  - A **computer with multiple CPU chips**.
  - Each CPU may have multiple cores.
  - Connected via system bus / interconnect.
  - Example: High-end servers with 4×64-core CPUs.
  - **Use Case:** Data centers, supercomputers → massive parallelism.

Multi-core vs Multi-processor Systems

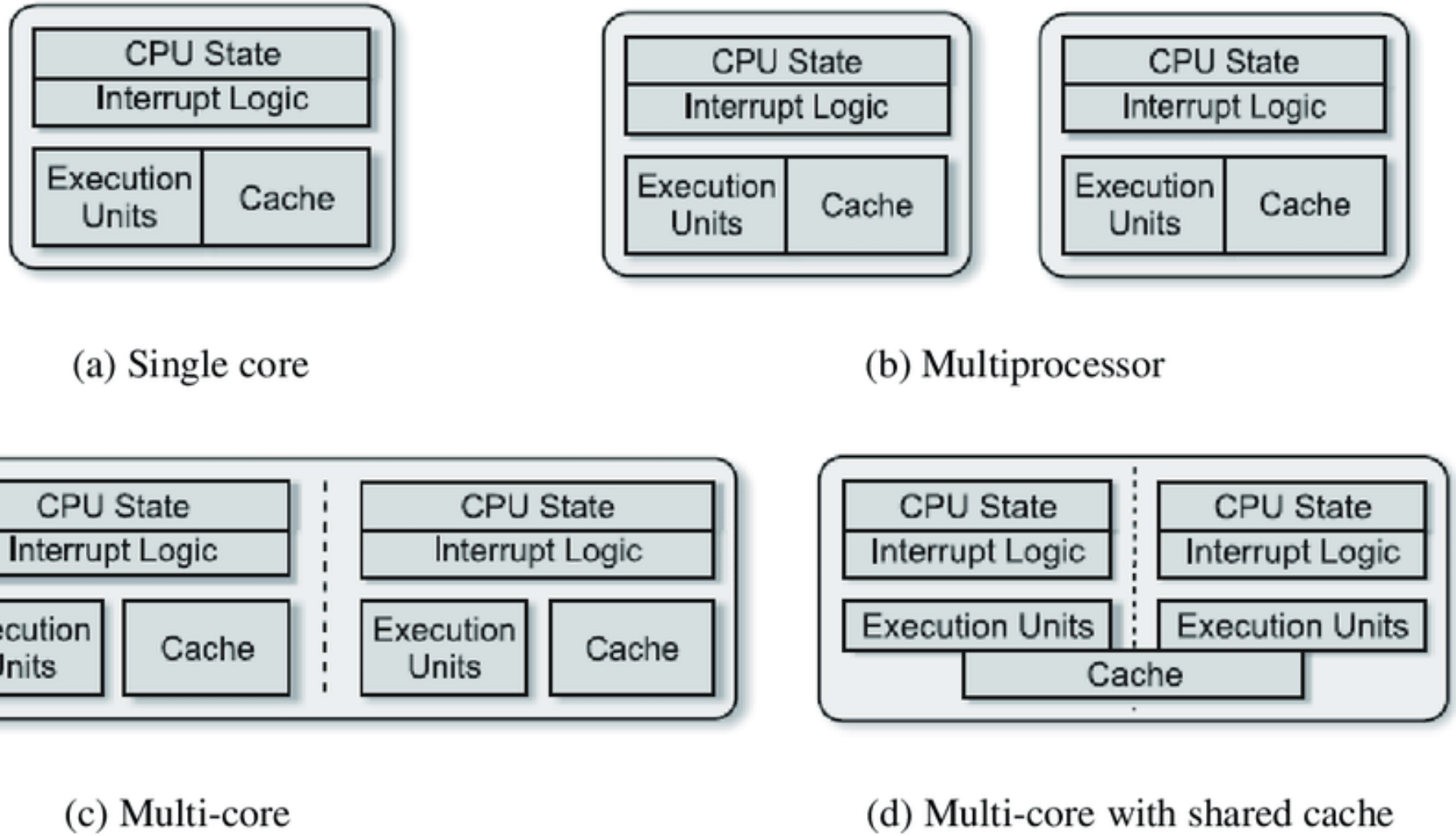


Multicore processor



Multiprocessor

Multi-core vs Multi-processor Systems



(a) Single core

(b) Multiprocessor

(c) Multi-core

(d) Multi-core with shared cache

What is Distributed Computing?

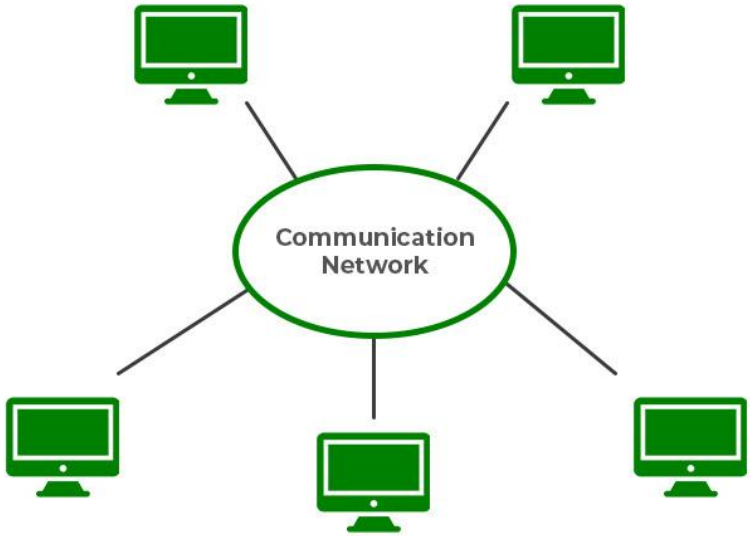
- Computation performed on multiple networked machines, which may or may not share memory.
  - Nodes perform subtasks independently.
  - Results combined at the end.
- Real-World Examples:
  - Google Search index across thousands of servers.
  - Netflix recommendation system.
  - Blockchain validation nodes.
- “What happens if one node fails in the middle of a computation? How would you recover?”

What is Distributed Computing?

- "What happens if one node fails in the middle of a computation? How would you recover?"
  - **Redundancy / Replication** – same task assigned to multiple nodes so if one fails, another result is available.
  - **Checkpointing** – system saves intermediate results; if a node fails, the computation restarts from the last checkpoint instead of the beginning.
  - **Task Reassignment** – failed task is automatically moved to another active node.
  - **Consensus Protocols** (e.g., Paxos, Raft, Byzantine Fault Tolerance) – ensure system correctness even with failures.

Distributed Computing (Definition)

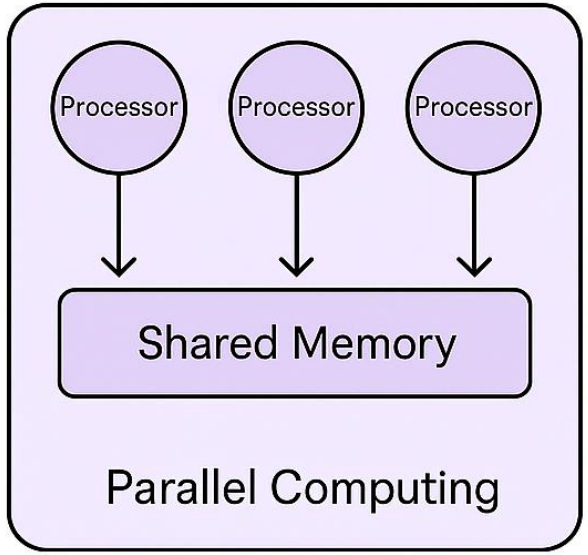
We define a **distributed system** as one in which ***hardware or software components*** located at **networked computers communicate and coordinate their actions only by passing messages**.



Parallel vs Distributed Computing

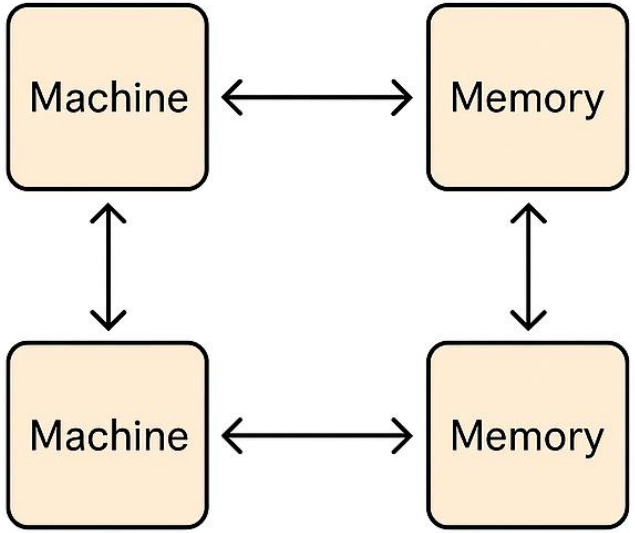
Parallel Computing

- Multiple processors
- Single machine
- Shared memory



Distributed Computing

- Multiple computers
- Network
- Distributed memory



Parallel vs Distributed Computing

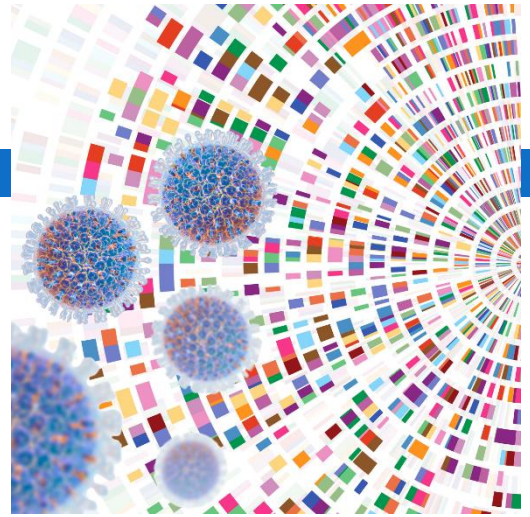
Aspect	Parallel Computing	Distributed Computing
Hardware	Single machine, multi-core/CPU/GPU	Multiple machines connected via network
Communication	Shared memory	Message passing / Network
Fault Tolerance	Low	High
Latency	Low	Can be high due to network delays
Scalability	Limited to cores in a single machine	High, can scale across clusters/data centers
Examples	GPU matrix computation, AI training	Google search, cloud apps, Netflix

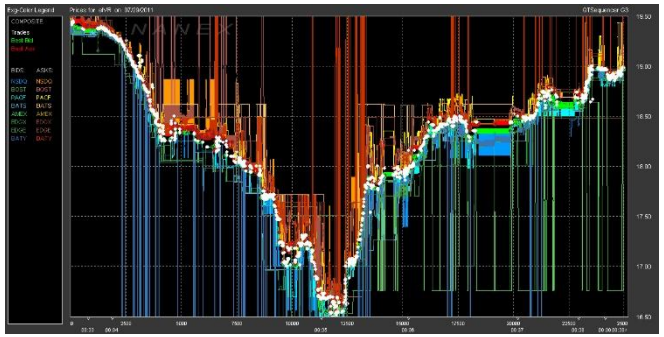
Why Use Parallel & Distributed Computing?

- Performance / Speed:
  - Solve computationally intensive problems faster by splitting workloads.
- Scalability:
  - Handle massive datasets or simulations.
- Real-Time Applications:
  - AI inference, AR/VR, autonomous vehicles.
- Cost Efficiency:
  - Using distributed cloud resources can be cheaper than a single supercomputer.
- Reliability / Fault Tolerance:
  - Distributed systems can continue functioning even if some nodes fail.

Examples

- AI & Deep Learning:
  - GPT models trained on thousands of GPUs in parallel.
- Graphics & Gaming:
  - Real-time 3D rendering requires thousands of calculations per frame.
- Scientific Research:
  - Climate modeling, genome sequencing.
- Finance & Business:
  - High-frequency trading, fraud detection.
- Cloud Computing:
  - Google Search, YouTube, AWS services.





Why Not Used – Challenges

- **Programming Complexity:** Writing parallel/distributed code is harder than sequential.
- **Debugging & Testing:** Concurrency bugs, deadlocks, and race conditions are difficult to identify.
- **Cost / Resource Limitations:** Hardware & energy costs may be prohibitive.
- **Communication & Synchronization Overhead:** Too much coordination can reduce performance.
- **Problem Suitability:** Not all problems can be parallelized (e.g., sequential algorithms).

“Can parallelism ever make a program slower than sequential execution? Why?”

Case Study – When Parallelism Fails

Small-scale matrix multiplication on a supercomputer → splitting tasks and combining results **takes longer than sequential computation**.

- Why does overhead reduce efficiency?
- How do we estimate **break-even problem size**?

- Students calculate approximate task sizes where parallelism becomes beneficial.

Case Study – When Parallelism Fails

- How do we estimate **break-even problem size**?
  - Break-even size is when:
  - $T_{sequential} \approx T_{parallel} = T_{compute} + T_{overhead}$
  - If the matrix is too small → $T_{overhead} > T_{compute}$ →parallelism is slower.
  - If the matrix is large enough → $T_{compute} \gg T_{overhead}$ →parallelism gives speedup.
- Students calculate approximate task sizes where parallelism becomes beneficial.
  - Sequential multiply: $O(n$^3) operations.
  - Overhead per processor (communication + sync): constant $O(n$^2) or fixed milliseconds.

Distributed System Trade-offs (1)

- **CAP Theorem**
- In distributed systems, you can only fully guarantee **2 out of 3**:
  - **Consistency (C):** All nodes see the same data at the same time.
  - **Availability (A):** Every request gets a response, even if some nodes fail.
  - **Partition Tolerance (P):** System continues to operate despite network failures.
- **Example:**
  - Banking system → Consistency + Partition Tolerance, sacrifices Availability.
  - Amazon shopping cart → Availability + Partition Tolerance, eventual Consistency.

Distributed System Trade-offs (2)

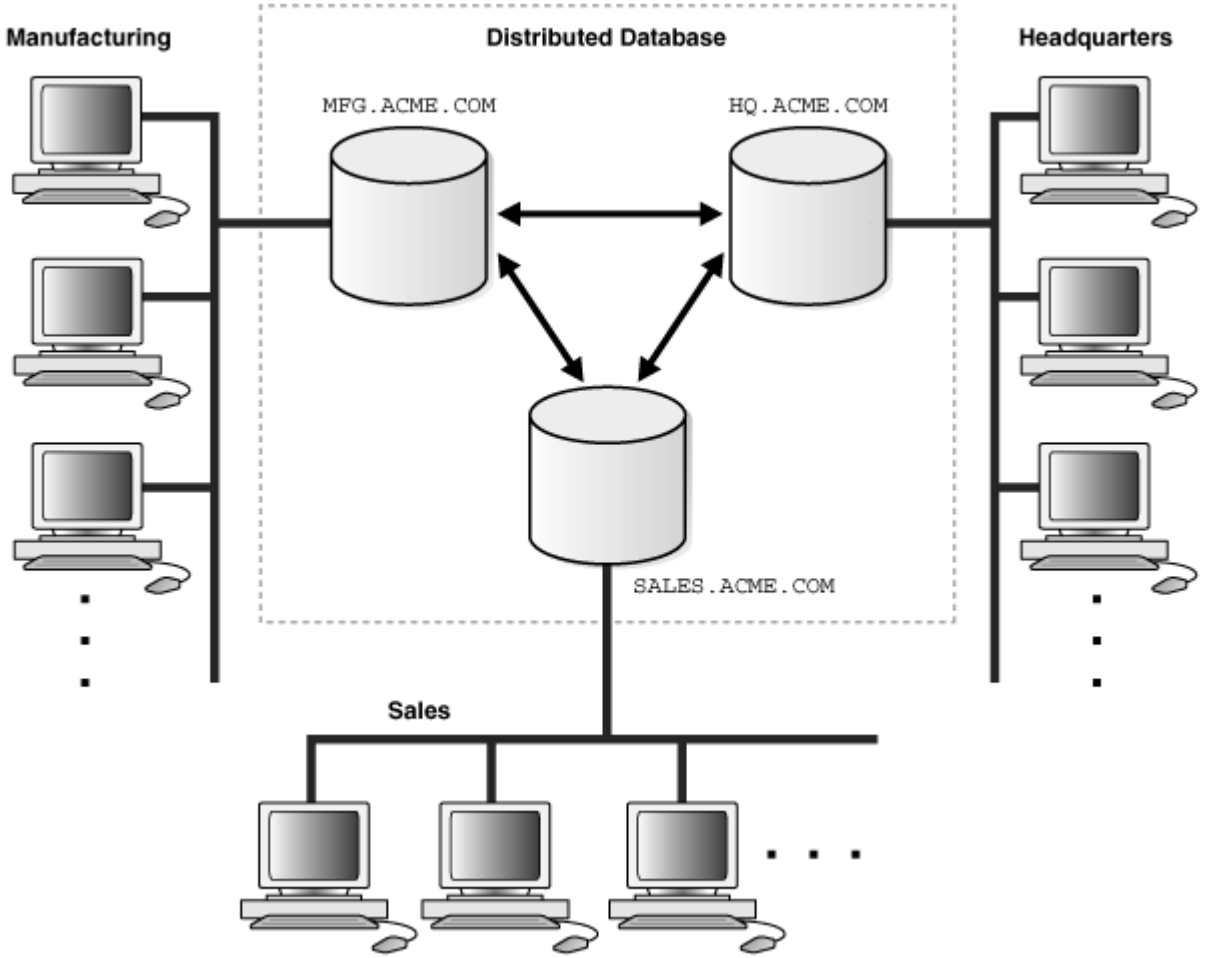
- **Latency vs Throughput**
- **Latency:** Time to process a single request.
- **Throughput:** Number of requests processed per unit time.
- Adding more nodes can **increase throughput**, but network communication introduces **latency**.
- **Example:** Distributed web search: more servers = faster total processing, but request coordination adds delay.

Distributed System Trade-offs (3)

- **Fault Tolerance vs Performance**
- **Fault Tolerance:** Replicating data across multiple nodes increases reliability.
- **Performance Cost:** Extra replication adds storage overhead and slows writes.
- **Example:** Netflix replicates across regions for reliability, but replication introduces synchronization delay.

A distributed database with 3 nodes.

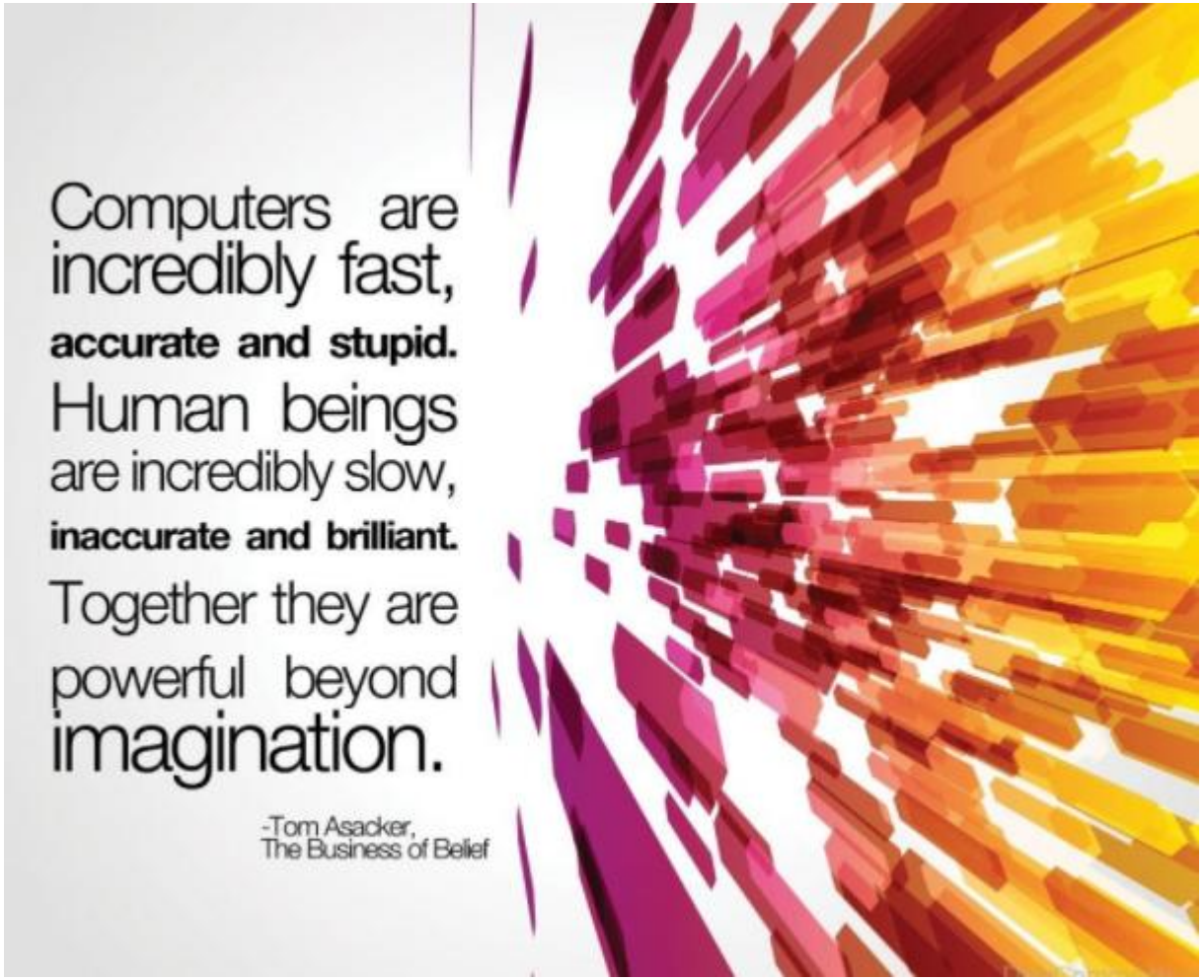
- If network between nodes fails, which CAP property do we sacrifice?
- If we add more replication for safety, how does performance change?
- Would you prefer low latency or high fault tolerance?



Summary

- Parallel:
  - Single machine, multiple tasks/data simultaneously.
- Distributed:
  - Multiple machines, tasks distributed across nodes.
- Why Used:
  - Speed, scalability, reliability, cost efficiency.
- Why Not Used:
  - Complexity, cost, synchronization overhead, unsuitable problems.

Thanks


Computers are incredibly fast, accurate and stupid. Human beings are incredibly slow, inaccurate and brilliant. Together they are powerful beyond imagination.
-Tom Asacker, The Business of Belief